plots Â¶

LOGGER `module-attribute` Â¶

```
LOGGER = getLogger(__name__)
```

quiet_mode `module-attribute` Â¶

```
quiet_mode = False
```

plot_calendar Â¶

```
plot_calendar(field_name, histogram_df, xaxis='date', yaxis='yearmonth', color='fr',
size='freq', color_scheme=None, target_field=None, target_value=None)
```

Function to plot a calendar-like plot of a time variable using Altair.

Recommended combinations of axis units are (xaxis="date", yaxis="yearmonth") and (xaxis="hours", yaxis="yearmonthdate").

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*
`xaxis` Â¶		time unit of the xaxis. Default: "hours". Recommended combinations of units are (xaxis="date", yaxis="yearmonth") and (xaxis="hours", yaxis="yearmonthdate")	`'date'`
`yaxis` Â¶		time unit of the yaxis. Default: "yearmonthdate".	`'yearmonth'`
`color` Â¶		color encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	`'fr'`
`size` Â¶		size encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	`'freq'`
`color_scheme` Â¶		Vega color scheme to be used. Available schemes can be found here: https://vega.github.io/vega/docs/schemes/. Default is None, meaning 'blues' for 'freq' and diverging 'blueorange' centered at the overall fraud rate for 'fr'.	`None`
			`None`

Name	Type	Description	Default
`target_field` Â¶		name of the target field. Must be given if color/size encoding contain 'fr'. Otherwise the given fraud rate encoding will be disabled.	
`target_value` Â¶		value to be considered as 'positives' labels. Must be given if color/size encoding contain 'fr'. Otherwise the given fraud rate encoding will be disabled.	`None`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_fraud_rate_numerical Â¶

```
plot_fraud_rate_numerical(field_name, histogram_df, target_field, target_value, nbins=50, fix_step=False, bar=True, logy=False, interactive=True)
```

Function for default fraud rate plotting of numerical variables using Altair.

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*
`target_field` Â¶		name of the target field.	*required*
`target_value` Â¶		value to be considered as 'positives' labels.	*required*
`nbins` Â¶		approximate number of bins for the desired histogram visualization. Default: 50	`50`
`fix_step` Â¶		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not	`False`

Name	Type	Description	Default
		fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	
bar Â¶		bool indicating whether to return a bar chart (True) or a line chart (False). Default: True.	`True`
logy Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
interactive Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart. If bar=False, this will be a layered Altair chart.

plot_fraud_rate_string Â¶

```
plot_fraud_rate_string(field_name, histogram_df, target_field, target_value, sort_by='fr', cat_threshold=None, vertical=True, logy=False, interactive=True)
```

Function for default fraud rate plotting of categorical/string variables using Altair.

Parameters:

Name	Type	Description	Default
field_name Â¶		field name.	*required*
histogram_df Â¶		Pandas DataFrame representation of a histogram.	*required*
target_field Â¶		name of the target field.	*required*
target_value Â¶		value to be considered as 'positives' labels.	*required*
sort_by Â¶		string indicating sorting type. Supported sorting are by frequency ('freq'), by fraud rate ('fr') and alphabetical ('alpha'). Default: 'fr'.	`'fr'`
			`None`

Name	Type	Description	Default
`cat_threshold` Â¶		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	
`vertical` Â¶		bool indicating whether to produce a vertical or a a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	`True`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart. Default: True.	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_fraud_rate_time Â¶

```
plot_fraud_rate_time(field_name, histogram_df, target_field, target_value, time_unit='yearmonthdate', time_type='T', logy=False, interactive=None)
```

Function for default fraud rate plotting of time variables using Altair.

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*
`target_field` Â¶		name of the target field.	*required*
`target_value` Â¶		value to be considered as 'positives' labels.	*required*
`time_unit` Â¶		a time unit supported by Vega-Lite. Recommended: "yearmonthdate" or "yearmonthdatehours". For more details: https://vega.github.io/vega-lite/docs/timeunit.html	`'yearmonthdate'`

Name	Type	Description	Default
`time_type` Â¶		Vega Data Type for time variable (default: "T"). Recommended "T" (="temporal") for high granularity time resolution, and "O" (="ordinal") for low cardinality time_unit like "month". For more details: https://vega.github.io/vega-lite/docs/type.html	`'T'`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
`interactive` Â¶		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	`None`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_numerical Â¶

```
plot_numerical(field_name, histogram_df, nbins=50, fix_step=False, normalize=None, breakdown_by=None, stack=False, logy=False, interactive=True, mini=False)
```

Function for default plotting of numerical variables using Altair.

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*
`nbins` Â¶		approximate number of bins for the desired histogram visualization. Default: 50	`50`
`fix_step` Â¶		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	`False`

Name	Type	Description	Default
`normalize` Â¶		bool (or None) indicating if the histogram is to be normalized. If set as True, the histogram is normalized to unity, otherwise the y-axis represents the actual counts. If set as True and breakdown_by is not None, the histogram of every breakdown value will be normalized to unity. Default is None: this means that the histogram will be normalized if breakdown_by is not None, otherwise it will not be normalized.	`None`
`breakdown_by` Â¶		string specifying a field to breakdown histograms by. Default is None: no breakdown.	`None`
`stack` Â¶		whether to stack histograms or not. Default: None. Only applies to the case where breakdown_by is not None.	`False`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`mini` Â¶		bool indicating if this is a mini-histogram.	`False`

Returns:

Type	Description
`unknown`	An Altair chart.

plot_string Â¶

```
plot_string(field_name, histogram_df, sort_by_freq=True, breakdown_by=None, cat_threshold=None, vertical=True, logy=False, interactive=True, mini=False)
```

Function for default plotting of categorical/string variables using Altair.

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*

Name	Type	Description	Default
`sort_by_freq` Â¶		bool indicating whether to sort by frequency, i.e. categories with higher frequency first. Otherwise, sort alphabetically (default: True).	`True`
`breakdown_by` Â¶		str specifying field to breakdown histograms by. Default is None: no breakdown.	`None`
`cat_threshold` Â¶		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	`None`
`vertical` Â¶		bool indicating whether to produce a vertical or a a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	`True`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart. Default: True.	`True`
`mini` Â¶		bool indicating if this is a mini-histogram.	`False`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_aggregation Â¶

```
plot_temporal_aggregation(df, time_field, field_name, aggregate='avg', time_resolution='1 day', show_points=False, interactive=True, zero_scale=True)
```

Function to process a PySpark DataFrame, computing a given aggregate function on a given field over time windows defined by a time resolution, and plotting it using a line plot.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	*required*
`time_field` Â¶		name of the PySpark DataFrame column associated to the time field.	*required*

Name	Type	Description	Default
`field_name` Â¶		name of the PySpark DataFrame column associated to the field used to calculate the aggregation over.	*required*
`aggregate` Â¶		aggregation function to be used. The available functions are "avg", "max", "min", "sum", "count". UDF created with pyspark.sql.functions.pandas_udf() may also be used. See https:/ /spark.apache.org/ docs/latest/api/python/pyspark.sql.html#pyspar k.sql.GroupedData for more details. (default: avg).	`'avg'`
`time_resolution` Â¶		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apac he.org/docs/latest/api/python/ pyspark.sql.html?highlight=window# module-pyspark.sql.functions for more details. (default: 1 day).	`'1 day'`
`show_points` Â¶		bool indicating whether to display the data points (default: False)	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`zero_scale` Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_boxplot Â¶

```
plot_temporal_boxplot(df, field_name, bar_size=10, interactive=True, zero_scale=True)
```

Function for default plotting a box plot of numerical or timestamp variables over time, using Altair.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pandas DataFrame with the aggregated data. The expected format is six columns. The first should be of type datetime and the remaining should contain the min, max, 25%, median and 75% values of the field of interest in a time window.	*required*
`field_name` Â¶		field name.	*required*

Name	Type	Description	Default
`bar_size` Â¶		numeric value used to specify the width of the bars (default: 10).	`10`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`zero_scale` Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_cardinality Â¶

```
plot_temporal_cardinality(df, time_field, field_name, show_cardinality_over_trx=True,
show_points=False, interactive=True, zero_scale=True)
```

Function for default plotting two line plots of the cardinality of a categorical variable over time, using Altair.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pyspark DataFrame with the aggregated data. The expected format is three columns. The first should be a datetime and the second and third should be numerical. The column names will be used as the axis titles.	*required*
`time_field` Â¶		field name for the temporal dimension.	*required*
`field_name` Â¶		field name.	*required*
`show_cardinality_over_trx` Â¶		bool display a second plot of the cardinality divided by the total number of transactions in that time period.	`True`
`show_points` Â¶		bool indicating whether to display the data points (default: False).	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`zero_scale` Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_lines Â¶

```
plot_temporal_lines(df, field_name, show_points=False, interactive=True, zero_scale=True, logy=F
alse)
```

Function for default plotting line plots of given numeric data points, over time, using Altair.

Multiple lines can be drawn representing different categories. It may be used to plot, for example, the values of an aggregation function (min, max, mean, ...) of numerical variables or the cardinality of a categorical variable over time.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pandas DataFrame with the data. The expected format is two or three columns. The first corresponds to the time dimension (x-axis) and should be of datetime type. The second corresponds to an arbitrary numerical quantity (y-axis), for instance a mean or a cardinality. If the third, and optional, column is passed, it will be used to draw multiple lines using Altair's color encoding and should be of type string. The column names will be used as the axis titles.	*required*
`field_name` Â¶		field name.	*required*
`show_points` Â¶		bool indicating whether to display the data points (default: False).	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`zero_scale` Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_points Â¶

```
plot_temporal_points(df, time_field, field_name, breakdown_field=None, interactive=True)
```

Function for plotting points given numeric data, over time, using Altair.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pandas DataFrame with the data.	*required*
`time_field` Â¶		name of the column associated to the time field.	*required*
`field_name` Â¶		name of the column associated to the numerical field.	*required*
`breakdown_field` Â¶		name of the column used as a breakdown field. The points will be color encoded by this field.	`None`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_quantiles Â¶

```
plot_temporal_quantiles(df, time_field, field_name, time_resolution='1 day', bar_size=10, interactive=True, zero_scale=True)
```

Function to process a PySpark DataFrame, computing the min, max and quantiles of a given field over time windows defined by a time resolution, and plotting it using a boxplot.

Parameters:

Name	Type	Description	Default
`df` Â¶		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	*required*
`time_field` Â¶		name of the PySpark DataFrame column associated to the time field.	*required*
`field_name` Â¶		name of the PySpark DataFrame column associated to the field used to calculate the quantiles over.	*required*

Name	Type	Description	Default
time_resolution Â¶		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apac he.org/docs/latest/api/python/ pyspark.sql.html?highlight=window# module-pyspark.sql.functions for more details. (default: 1 day).	`'1 day'`
bar_size Â¶		numeric value used to specify the width of the bars (default: 10).	`10`
interactive Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
zero_scale Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_temporal_top_n Â¶

```
plot_temporal_top_n(df, time_field, field_name, top_n=5, time_resolution='1 day', show_points=False, interactive=True, zero_scale=True)
```

Function for plotting a line plot for the counts of the top-n most frequent categories of a given string field over a time window defined by a time resolution.

It shows the evolution over time of a given quantity for each category, using Altair.

Parameters:

Name	Type	Description	Default
df Â¶		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name. It corresponds to the time dimension (x-axis) and should be of datetime type. The other should have the value of field_name as the column name. It contains categorical values of str type and corresponds to the categorical dimension (color encoding).	*required*
time_field Â¶		name of the PySpark DataFrame column associated to the time field.	*required*
field_name Â¶		name of the PySpark DataFrame column associated to the string field.	*required*

Name	Type	Description	Default
`top_n` Â¶		the number of categorical values, with highest count, that should be used in the aggregation.	`5`
`time_resolution` Â¶		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apac he.org/docs/latest/api/python/ pyspark.sql.html?highlight=window# module-pyspark.sql.functions for more details. (default: 1 day).	`'1 day'`
`show_points` Â¶		bool indicating whether to display the data points (default: False)	`False`
`interactive` Â¶		bool indicating whether to call the function interactive() on the Altair chart (default: True).	`True`
`zero_scale` Â¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	`True`

Returns:

Type	Description
`unknown`	an Altair chart.

plot_time Â¶

```
plot_time(field_name, histogram_df, time_unit='yearmonthdate', time_type='T', breakdown_by=None, logy=False, interactive=None, mini=False)
```

Function for default plotting of time variables using Altair.

Parameters:

Name	Type	Description	Default
`field_name` Â¶		field name.	*required*
`histogram_df` Â¶		Pandas DataFrame representation of a histogram.	*required*
`time_unit` Â¶		a time unit supported by Vega-Lite. Recommended: "yearmonthdate" or "yearmonthdatehours". For more details: https://vega.github.io/vega-lite/docs/timeunit.html	`'yearmonthdate'`

Name	Type	Description	Default
`time_type` Â¶		Vega Data Type for time variable (default: "T"). Recommended "T" (="temporal") for high granularity time resolution, and "O" (="ordinal") for low cardinality time_unit like "month". For more details: https://vega.github.io/vega-lite/docs/type.html	`'T'`
`breakdown_by` Â¶		str specifying field to breakdown histograms by. Default is None: no breakdown.	`None`
`logy` Â¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	`False`
`interactive` Â¶		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	`None`
`mini` Â¶		bool indicating if this is a mini-histogram.	`False`

Returns:

Type	Description
`unknown`	an Altair chart.

`set_quiet_mode` Â¶

```
set_quiet_mode(mode)
```